
Linux and Android Security Models Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *December 5th, 2019*

Introduction

1. Every exploitability analysis starts with the mitigations that live *inside* one process (NX, canaries, ASLR, RELRO, CFI). This module covers the layer above: once you have code execution as *some* process, what does the operating system still stop you from doing? That answer — the OS trust model.

The Reference Monitor and the Trust Model

1. The academic model: a **reference monitor** mediates every access to every object and is (a) **complete** (no access bypasses it), (b) **tamperproof**, and (c) **small enough to verify**. Real systems approximate each property; knowing which one a mechanism gives up predicts its bug class.
2. The subject/object/operation triple is the audit unit:
 - (a) **Subject**: who is asking. Linux: the `struct cred` on the task (uid/gid sets, capabilities) plus the LSM label. Android: the app UID plus the SELinux domain.
 - (b) **Object**: what is being reached. Linux: an inode (owner, mode, ACLs, xattr-carried label). Android: a file under `/data/data/<pkg>` with an `app_data_file` label, or a Binder service.
 - (c) **Operation**: read, write, execute, `ioctl`, bind, `ptrace`, mount, Binder transaction, and so on.
3. **Preventing exploitation**. Model deficiencies / security bugs can be 1. patched, 2. mitigated, or 3. attack surface minimized. These are the primary mechanisms used by developers to fix models. Getting updates to systems can be politically complicated, and lead to patch gaps.
4. **Mechanism versus policy**. The mechanism is code (VFS permission checks, LSM hooks, the Binder driver's checks, `seccomp`'s filter interpreter). The policy is data (the SELinux binary policy, an AppArmor profile, an Android app's permission set, a `seccomp` filter program). Most real-world "model bugs" are policy bugs — an over-broad rule, a permissive domain, a world-writable path — and policy is configuration, so it ships unreviewed and unfuzzed.
5. **Defense in depth** is the design principle Android leans on hardest, and the reason a single model bypass is usually not game over. It is also why the layers must

be audited *jointly*: the interesting bugs live at the seams (a uid boundary crossed by an fd handoff, a label check skipped for a shared-memory buffer).

6. **Complete mediation** is the property to test first, on any target. Ask for each privileged operation: which single check implements it, and can the object change between the check and the operation? That question is the whole of the next section's bug catalogue.
7. The three properties fail in practice in three named ways, and each failure is an audit method of its own: completeness fails through an unchecked second path to the same object (a second `ioctl` entry, a second socket, a `debugfs` alias); tamperproofness fails through policy a governed subject can write (an over-broad `allow`, a writable policy file, a world-writable device node); and verifiability fails through a mechanism too large to read (a BPF-LSM policy program, a driver's own permission check).
8. The one-sentence definition to carry: a memory-safety bug is an inconsistency in memory, and a model bug is an inconsistency about authority.
9. The one-sentence definition to carry: a memory-safety bug is an inconsistency in memory, and a model bug is an inconsistency about authority.

Linux: Subjects, Credentials, and DAC

1. The subject is `current->cred` (`include/linux/cred.h`), and it carries four uid/gid identities, not one:
 - (a) **real** (uid/gid): who invoked the process.
 - (b) **effective** (euid/egid): used for most permission checks, including signal delivery and file creation.
 - (c) **saved-set** (suid/sgid): the value `setuid` can return to. A `setuid-root` program that calls `setuid(getuid())` has *not* dropped root: the saved-set uid is still 0, so `setuid(0)` restores it. The complete drop is `setresuid(getuid(), getuid(), getuid())`, performed before any attacker-influenced code runs.
 - (d) **filesystem** (fsuid/fsgid): used by the VFS for file access, so a daemon (an NFS server, say) can act for a user without exposing the user to signals. Auditing consequence: a program that drops only `euid` and forgets `fsuid` still reads and writes files as the old identity.
2. Credentials also carry the capability sets and the LSM security blob (the SELinux SID, the AppArmor label). Two rules follow, and both are recurring bug sources:
 - (a) **The drop must be complete**: uid, euid, saved-set, fsuid, gids, supplementary groups, capabilities, and the LSM label (a domain transition back to an unconfined domain is a full privilege regain). Auditing recipe: for every `setuid` family call, list which fields change and what remains.
 - (b) **The identity must be the kernel's**: a service that reads a uid from a message (a Binder parcel, a netlink header, a socket credential the peer set) rather than from the kernel's own record of the sender is a confused deputy.

- i. Linux's equivalents of the attested identity are `SO_PEERCREDS`, `SCM_CREDENTIALS`, and `getsockopt(SO_PEERSEC)` for the peer's LSM label.
 - ii. Android's is `Binder.getCallingUid()`.
- 3. DAC is the classic layer: owner/group/other bits checked against the inode, with a POSIX ACL (an `system.posix_acl_access xattr`) overriding the mode bits when present, and the directory sticky bit restricting deletion. The check is `inode_permission()` → `generic_permission()`, reached from `may_open()` in the path walk. Root's bypass is `CAP_DAC_OVERRIDE` (write) and `CAP_DAC_READ_SEARCH` (read/search).
- 4. The DAC details that show up in real bugs:
 - (a) **umask applies at creation, not at use:** a file created 0666 under umask 022 is 0644 forever; the model question is what mode the *creator* chose, not what the caller hoped for.
 - (b) **Ownership, not path, is the boundary:** a privileged process writing to a path under attacker influence has the attacker's choice of object if the directory is attacker-writable — the path-race bug class.
 - (c) **Mount options are policy too:** `nosuid`, `nodev`, `noexec`, and the read-only remount are the cheap, blunt mitigations; a bind mount that re-enables `suid` on a subtree is a model bug.
 - (d) **Special files:** a `/dev` node's mode and label are the whole policy for that driver; `/proc` and `/sys` entries are often 0644 with privileged *content*, so the write path's own checks are the only gate (`kptra_restrict`, `dmesg_restrict`, and the `perf_event_paranoid` sysctls exist precisely because file mode was the wrong gate).

Linux: Capabilities

1. Root's authority is not one bit; it is a set of ~40 capabilities, each guarding a family of operations. The sets on a task are **permitted** (the ceiling), **effective** (currently checked), **inheritable**, **bounding** (a hard ceiling across exec), and **ambient** (a newer set that lets a non-root binary keep capabilities across exec without file capabilities).
2. The capability table to know by heart for auditing (the authority each grants an attacker):

Capability	Authority
<code>CAP_DAC_OVERRIDE</code>	bypass file read/write/execute bits
<code>CAP_DAC_READ_SEARCH</code>	bypass file read and directory search
<code>CAP_FOWNER</code>	<code>chmod/chown</code> on others' files
<code>CAP_SETUID</code> , <code>CAP_SETGID</code>	assume any uid/gid; often the shortest path to root
<code>CAP_SYS_PTRACE</code>	debug/inject any process
<code>CAP_SYS_ADMIN</code>	mount, unshare, many driver ioctls — the de facto root
<code>CAP_SYS_MODULE</code>	load kernel code
<code>CAP_NET_ADMIN</code> , <code>CAP_NET_RAW</code>	configure interfaces; sniff/spoof
<code>CAP_SYS_CHROOT</code> , <code>CAP_SYS_TIME</code>	chroot escape; clock manipulation

3. Checks are `capable(CAP_X)` (initial user namespace) and `ns_capable(ns, CAP_X)` (a specific namespace). The namespaced form is the reason a user namespace is a privilege boundary in itself: `CAP_NET_ADMIN` inside your own netns is real authority over that namespace, and a bug reachable from there is a kernel bug.
4. Two privilege transitions to audit, both of which produce retained authority when done incompletely:
 - (a) **setuid-root binaries**: the `exec` sets `euid 0` and the permitted set to full; the program must drop deliberately, and `PR_SET_NO_NEW_PRIVS` is what stops a later `execve` from re-raising privilege.
 - (b) **file capabilities** (`security.capability xattr`, `setcap/getcap`): a non-root binary can start with a chosen capability in its effective set. Audit both: which capabilities the file grants, and whether the program drops the ones it does not need.
5. Enumeration: `capsh --print`, `/proc/self/status` (`CapInh`, `CapPrm`, `CapEff`, `CapBnd`, `CapAmb`), `getcap -r / 2>/dev/null`. The same files are the audit trail for a privilege-drop bug: a process that claims to have dropped privilege but still shows a non-empty `CapEff` has not.

Linux: MAC and the LSM Framework

1. The kernel exposes one hook list for all security modules: `include/linux/lsm_hooks.h`, called from `security/security.c` at the chokepoints — enforced at syscall gateways and other kernel chokepoints.
2. Hooks that matter to a VR audit, with what they gate:
 - (a) `inode_permission`, `inode_create`, `inode_unlink`: the file DAC/MAC pair.
 - (b) `file_open`, `file_ioctl`, `file_mmap`, `file_receive`: the driver surface, gated per verb.
 - (c) `mmap_file`, `file_mprotect`: executable mappings, the $W \oplus X$ gate.
 - (d) `task_kill`, `ptrace_access_check`, `task_prctl`: process control.
 - (e) `socket_bind`, `socket_connect`, `socket_sendmsg`: network labelling.
 - (f) `bprm_check_security` (`exec`), `kernel_load_data` (module load), `settime`, `sysctl`.
3. The modules and how they differ (the stacking order matters: `capability` is always first, so DAC-style checks run before any MAC module):
 - (a) **SELinux**: label-based, type enforcement, complete mediation. A subject label is `user:role:type:level`; the policy grants allow rules between types; `type_transition` (Android: `domain_auto_trans`) assigns the label of a process at `exec`. Per-domain permissive flags and booleans exist, and `neverallow` rules are compile-time invariants checked when the policy is built.
 - (b) **AppArmor**: path-based, per-program profiles, `complain` (log-only) versus `enforce` mode. Weaker against path races and hardlinks by construction; easier to read.

- (c) **Smack** and **TOMOYO**: label-based and path-based respectively; rarer, but they show up in embedded and IoT firmware.
 - (d) **BPF-LSM**: the newer dynamic module; policy is a BPF program, so the policy is now code with its own memory-safety bug class.
4. Where the code lives: `security/security.c` (hook caller), `security/selinux/hooks.c` (the SELinux implementations, e.g. `selinux_inode_permission`), `security/apparmor/`, and the policy in `system/sepolicy` plus `device/*/sepolicy` on Android.
 5. The audit surface of the MAC layer:
 - (a) **Denials are the debugging oracle**: read them from the kernel log, from the audit log, and from the AVC cache stats. `audit2allow` then prints the allow rule that would have permitted the denial — which is exactly the rule you then argue is over-broad.
 - (b) **Policy as data**: the policy inspection tools, the compiled `policy.<ver>` file, and the source policy tree are all readable on an unlocked device. Diff two vendor images' policies to find what changed — the audit-queue rule applies to policy as much as to code.
 - (c) **Permissive domains are a bug class of their own**: a domain in permissive mode logs and allows everything; a shipped permissive domain is a model vulnerability that a device-security diff will reveal.
 - (d) `neverallow` violations are caught at policy build time, so a vendor that bypasses the check (or builds with it disabled) ships a policy the AOSP authors believed impossible.

Linux: Namespaces, Containers, and seccomp

1. Namespaces are the isolation mechanism; they are not by themselves a security boundary. The boundary is the *combination* of namespaces, cgroups, capability drops, seccomp, and MAC.

Namespace	Isolates
<code>mnt</code>	mount table (what paths exist)
<code>pid</code>	process id space (what can be seen/killed)
<code>net</code>	interfaces, routes, iptables, sockets
<code>ipc</code>	SysV/POSIX message queues, shared memory
<code>uts</code>	hostname and domain name
<code>user</code>	uid/gid mapping, capability scope
<code>cgroup</code>	cgroup view; <code>time</code> (5.6+) clock offsets

2. The user namespace is the interesting one for VR: it is the only namespace an unprivileged user may create, and creating one grants a full capability set *scoped to that namespace*. `CAP_SYS_ADMIN` in a user namespace is real authority over the objects in that namespace, and any kernel bug reachable from that authority (mounting filesystems, `ioctl` paths, driver code that only checks `ns_capable`) is now reachable unprivileged. The 2016–2017 wave of container escapes was largely this class.

3. Container escape as a model question: enumerate what the container runtime *gave* the process (which namespaces, which capabilities, which seccomp profile, which MAC profile, which mounts) and find the one it assumed was covered by another layer. Recurring shapes:
 - (a) a mounted host path (the runtime's own socket, `/proc`, a device node) inside the container;
 - (b) `CAP_SYS_ADMIN` retained in the container's user namespace, plus a filesystem or driver bug reachable from it;
 - (c) a leaked host file descriptor (an fd-lifetime bug, at model level);
 - (d) the shared kernel itself: namespaces do not change that all containers run one kernel, so the kernel attack surface is the container's surface too.
4. **seccomp-bpf** is an allowlist at the syscall boundary: a BPF program over `struct seccomp_data` decides per syscall. The fields are `nr`, `arch`, `instruction_pointer`, and six raw args.
 - (a) Actions: `SECCOMP_RET_KILL_THREAD/KILL_PROCESS`, `TRAP`, `ERRNO`, `TRACE`, `LOG`, `ALLOW`. A default of `ALLOW` with a blocklist is the wrong shape: the correct model is default-deny with an explicit allow list.
 - (b) The filter sees *values*, not pointees. An argument check on a pointer argument (a path, a buffer) validates the pointer, not the bytes; the kernel copies after the filter runs. That is a check-vs-use window by construction, not a bug.
 - (c) The filter must check `arch`: without it, a 32-bit process on an x86-64 kernel can invoke the x86-32 syscall with the same number as a blocked x86-64 one. The syscall-number tables differ between architectures, so an `nr`-only filter is bypassable.
 - (d) `SECCOMP_RET_TRACE` hands control to a tracer, and the tracer's own checks become part of the model (the `ptrace` substrate). `no_new_privs` is required for filter installation by unprivileged processes precisely so that a filter cannot be outlived by a `setuid` transition.
5. **Landlock** (5.13+) and **Yama** (`ptrace_scope`) are the newer unprivileged-facing LSMs; Yama's `sysctl` is the reason a non-root attacker cannot simply attach to a sibling process — relevant to every remote-attach workflow.
6. Enumeration: `seccomp-tools` dumps a filter, `/proc/<pid>/status` reports the mode, and `bpftrace` can watch its decisions. The recipes are collected in **Enumerating the Model** below. The filter program itself is attacker-relevant *policy*: read it before assuming the sandbox is complete.

Android: the Sandbox and the Permission Model

1. Android is a Linux kernel plus a userspace whose model is *per-app least privilege, layered*. The layers, outermost first, and what each is responsible for:
 - (a) **UID sandbox (DAC)**: each app is installed with a unique uid (10000+); its data directory `/data/data/<pkg>` is owned by that uid and mode 0700. Two

apps cannot read each other's data by DAC alone. The same mechanism isolates apps from the system's own files.

- (b) **SELinux (MAC)**: every process runs in a domain, every file has a label, and the policy grants the small set of accesses each domain needs. This is the layer that survives a DAC misconfiguration, and the layer that bounds what a compromised app can reach.
- (c) **Permissions (userspace policy)**: the manifest-declared permission set, checked by framework code, not by the kernel. A permission is a policy over the uid boundary; it is not enforced at a chokepoint.
- (d) **Verified boot, FBE, Keystore**: integrity and secrecy of the data the model operates on, so that a model bypass does not automatically yield persistent data.

2. The permission model in detail, because most Android model bugs live here:

- (a) **Protection levels**: normal (granted at install, no prompt), dangerous (runtime prompt, revocable), signature (only same-signer apps), signatureOrSystem (deprecated), privileged (only /system/priv-app). The level determines who can hold the permission at all.
- (b) **Runtime permissions** (Android 6+): a dangerous permission is granted or revoked per-app at runtime; the app must handle revocation mid-run. Model consequence: checkPermission results are *state*, and state changes — the check-vs-use shape again.
- (c) **Enforcement point**: framework code calls checkCallingPermission() (or its enforce variant), which resolves through ActivityManager using the caller's uid from the Binder transaction. The audit question is always: which uid did this check use?
- (d) **The “or self” footgun**: checkCallingOrSelfPermission() checks the caller *or the calling process's own permission*.
 - i. Correct for a service deciding its own capability; wrong for a service enforcing on behalf of a caller.
 - ii. A service that uses the OrSelf variant for a caller-authorization decision has granted its own permissions to every caller: the canonical Android confused deputy.
- (e) **App ops** are a second, runtime-revocable gate on top of permissions (camera, location, overlay). An app can hold the permission and still be denied by the app op; auditing either one alone gives the wrong answer.
- (f) **Exported components and grants**: each is a policy escape hatch and each is an audit target.
 - i. android:exported and intent filters: which components an app may reach with no permission at all.
 - ii. FLAG_GRANT_READ_URI_PERMISSION and the content-provider <path-permission> rules: deliberate, revocable, uid-bound exceptions to the DAC layer.
 - iii. Pending intents: whose sender identity the receiver sees, and whether the receiver can be chosen by the sender.

3. Process identity facts worth committing to memory: `Process.myUid()` is the process's own uid; `Binder.getCallingUid()` is the kernel-attested uid of the caller of the current transaction; `getCallingPid()` likewise. Isolated processes (`isolatedProcess`) get a fresh uid and no permissions; `sharedUserId` makes several packages share one uid and therefore one sandbox (legacy, discouraged).
4. The storage model changed the boundary: pre-Android 10, external storage was one shared world-writable filesystem with group-based access (`sdcard_rw`); Android 10+ scoped storage gives each app a synthetic view of only its own files, with media access mediated by `MediaProvider`. Audit both eras by their actual mechanism, not by the permission name.

Android: SELinux Domains and Binder

1. Android runs SELinux in enforcing mode since 5.0 with a policy that is *part of the platform image*. The policy files are the model, and they are auditable:
 - (a) `file_contexts`: path (or regex) → label. Every file's label comes from here at first labelling; an unlabelled file (a file created on a filesystem whose label the policy does not name) gets the filesystem's default label — an audit question.
 - (b) `seapp_contexts`: uid, `seinfo` (from the package's signature/privilege), and process name → domain. This is the join between the DAC layer (the uid) and the MAC layer (the domain): the app's uid determines its domain, which is why the uid sandbox and the domain cannot be reasoned about separately.
 - (c) `property_contexts`: property name → label, gating `setprop` by domain.
 - (d) Domains: `init`, `zygote`, `system_server`, `untrusted_app`, `untrusted_app_<N>` (per-app domains for isolated processes), `priv_app` (privileged preinstalled apps), `platform_app` (apps sharing the platform signature), `isolated_app` (no permissions at all), plus one per HAL and vendor service.
 - (e) `neverallow` rules are the invariants of the policy: they state what no shipped device may ever allow. A vendor policy that adds an `allow` in violation is caught by the AOSP build's compatibility check; a vendor that ships its own policy build with the check weakened is a real-world model bug.
2. **Binder is where the two layers meet**, and where most Android model bugs live:
 - (a) A transaction carries a kernel-attested sender uid/pid (`binder_get_calling_uid`) — services are supposed to use it and never a uid from the parcel.
 - (b) SELinux gates Binder itself: `binder_use` (may a domain use Binder at all), `binder_call` (domain → domain, i.e. may this app call this service), and `binder_transfer` (may this domain receive this class of fd over Binder — the check that makes fd-passing grants safe).
 - (c) `servicemanager` is the lookup broker and is itself labelled; `service_manager` checks `add/find` per domain, so the set of services an app can even *name* is policy.

- (d) Audit recipes: `service list`, `dumpsys -l`, `dumpsys activity services`, and the SELinux side with `ls -Z` on the service's own files and `ps -AZ` for domains. The kernel-side bugs in the Binder driver itself are ordinary kernel attack surface (CVE-2019-2215 is a Binder UAF): the model decides *who may call*; the driver's parser decides *what the bytes do*.
- 3. The vendor/system boundary (Treble) is a model boundary: `vndbinder` separates vendor services from framework services, and the policy is split into `system` and `vendor` parts that may not be merged. Vendor HALs are written by the SoC vendor, ship in the vendor image, and are reachable from the framework — which makes them the Android equivalent of vendor drivers in a kernel.

Android: Verified Boot, Encryption, and Keystore

1. **Verified boot (AVB2.0):** a chain of hash/signature verification from the hardware root of trust (boot ROM, keyed by e-fuses) through the bootloader, `vbmeta`, `boot`, `system`, and `vendor`. `dm-verity` enforces the integrity of the read-only partitions at block level with a Merkle tree, so a modified block fails verification on read.
 - (a) Boot states: green (locked, verified), yellow (locked, custom key), orange (unlocked), red (verification failed). Readable as `ro.boot.verifiedbootstate` and `ro.boot.flash.locked`.
 - (b) Model consequence: an unlocked bootloader is a model bypass, not a bug — it lets you replace the *policy* (a permissive SELinux build, an `su` domain, a modified `init.rc`). This is exactly what the rooting walkthrough does, and why `adb disable-verity` requires an unlocked bootloader.
 - (c) Rollback protection: the rollback index in `vbmeta` plus anti-rollback fuses prevent installing an older, vulnerable image. Downgrade is a model attack with its own name because the model assumes the patch cannot be undone.
2. **File-based encryption:** device-encrypted (DE) storage is available before unlock, credential-encrypted (CE) storage needs the user credential; `vold` and the kernel's `fsencrypt` manage the keys, which live in the TEE. The model question: which data is readable at which boot stage, and what does an offline attacker with the flash chip get.
3. **Keystore/Keymaster/StrongBox:** app keys are stored in the TEE (TrustZone) or a secure element and never leave it; the app gets an opaque handle and asks the service to sign/decrypt. Consequences:
 - (a) The key is not extractable by design, so the attack surface is the *service interface and its callers*, not the key material. Trustlets and TEE drivers are the model's own attack surface, and the vendor-driver audit discipline applies to them.
 - (b) Attestation binds a key to a device state; an attacker who can influence the attestation (a compromised TEE, a patched boot state) defeats a model that trusts it.
 - (c) Key destruction semantics are part of the model: a key that survives a factory reset, or a key that is backed up, is a persistence primitive.

4. Layering is the point: verified boot protects the code, FBE protects the data at rest, Keystore protects the keys, SELinux bounds the running processes, the uid sandbox bounds the apps, and the permission model bounds the framework calls. An attacker who owns any single layer still has to cross the others, which is why the *seams* (Binder fd passing, shared files, exported components) are the highest-value audit targets.

Auditing the Model

1. Model bugs fall into three families, and they need different audit methods:
 - (a) **Enforcement-code bugs:** memory-safety bugs in the code that implements the model (the LSM hook implementations, the Binder driver, the permission-check helpers, the seccomp filter verifier). These are ordinary memory-safety bug patterns, at a chokepoint. High value because the chokepoint runs with authority.
 - (b) **Policy bugs:** an over-broad allow, a permissive domain, a world-writable file, a `signatureOrSystem` permission, a `checkCallingOrSelfPermission` misuse, a seccomp default-allow. No sanitizer will find these. Audit method: read the policy as data, diff it against the AOSP baseline, and ask what the widest reachable access is.
 - (c) **Seam bugs:** the interfaces between layers, where each layer believes the other enforced the rule. The Binder uid, the passed file descriptor, the shared buffer, the label derived from a uid, the grant that outlives the check. This is where the confused deputy lives.
2. The confused deputy checklist, applied to any service that acts for a client:
 - (a) Who is the deputy, and what authority does it hold that the client does not?
 - (b) Which part of the request is attacker-controlled (a path, a uid, a flag, a buffer, a handle)?
 - (c) Does the check apply to the same object the operation uses? If the object is named twice, or resolved twice, it is a check-vs-use bug (a path race, in authorization form).
 - (d) Is the identity the kernel-attested one (`Binder.getCallingUid()`, `SO_PEERCREC`) or a value the client supplied?
 - (e) What is the worst operation the deputy can be made to perform, and on which object? That pair is the primitive, and it is assessed exactly like a memory-corruption primitive: what does it grant, and what must the attacker control to reach it.
3. Enumeration recipes to keep in the notes. All of them are read-only, and all of them are how you establish the model on a target:

Listing 1: Model enumeration recipes (Linux host / Android device / the policy files).

```

1  # Linux host
2  id; capsh --print; grep Cap /proc/self/status
3  getcap -r / 2>/dev/null; getenforce; ls -Z; ps -eZ
4  seinfo; sesearch -A; ausearch -m avc; dmesg | grep avc
5  mount | grep -E 'nosuid|noexec|nodev'
6  unshare -Ur id; seccomp-tools dump ./target
7
8  # Android device
9  adb shell id; getenforce; ls -Z /data/data; ps -AZ
10 cmd package list permissions -g -d; dumpsys package <pkg>
11 service list; dumpsys -l
12 getprop | grep -E 'ro.boot|ro.build'
13 su -c cat /proc/self/attr/current
14
15 # the policy files
16 #   system/sepolicy in AOSP, sepolicy/ in a vendor image,
17 #   and the compiled policy.<ver> on the device

```

4. Diff two devices' policies before assuming a behaviour is intended.
5. The audit-queue rule, restated for policy: mainline policy is reviewed and its neverallow invariants are checked at build time; vendor and OEM policy is neither, so the first question about any surprising allow is "which vendor added it, and what did that vendor's other code need it for?"
6. Reporting a model bug uses the standard report shape: the subject and its authority, the object and its intended rule, the enforced rule, the bypass, and the primitive. The difference is that the "reproducer" is a sequence of operations with identities (an app calling a service, a process reaching a node), not a malformed file.
7. A short worked example, to fix the shape. Android `system_server` runs in its own domain with broad authority and is called by every app. Suppose a service it exposes performs `checkCallingOrSelfPermission(P)` before writing a caller-supplied path. The subject is the app, the deputy is `system_server`, the object is any file the deputy can write, the intended rule is "only a caller holding P may cause the write", and the enforced rule is "any caller may cause the write" because `OrSelf` grants the deputy's own permission to every caller. The primitive is an arbitrary-file-write as `system_server`; the policy that should have caught it is the SELinux allow for the service's domain, which is irrelevant here because the write happens with the deputy's authority, not the caller's. That is the whole lesson of the confused deputy in one paragraph: *the MAC layer does not save a service that checks the wrong subject*.
8. Public examples worth naming, one per family:
 - (a) Enforcement-code bugs: the Binder driver's own use-after-free (CVE-2019-2215), the futex requeue bug (CVE-2014-3153), Dirty COW (CVE-2016-5195),

and the recurring `io_uring` and `netfilter` lifetime bugs. These are audited exactly like any other memory-safety bug.

- (b) Policy bugs: shipped permissive domains, `signatureOrSystem` permissions, world-writable device nodes, and vendor `allow` rules added for a single app. These are found by reading policy as data, not by fuzzing.
 - (c) Seam bugs: the confused deputy family (a service acting on a caller-supplied path), an fd-passing grant that outlives the check, and an Android `checkCallingOrSelfPermission` misuse. These are found by enumerating the interfaces between layers.
9. Reporting a model bug adds one paragraph to the standard report format: *which layer was supposed to enforce the rule, which layer actually did, and what the difference grants*. The reproducer is a sequence of operations with identities, and the exploitability statement assesses the primitive against the other layers that still stand.

Conclusion

The security model is the layer above the process. The mitigation map answers “what does this memory bug grant inside my process?”; this module answers “what may that process do next, and who will let it?” Two rules to leave with. First, enforcement bugs are memory-safety bugs at a chokepoint and are audited like any other code; policy bugs are data bugs that no fuzzer will find and must be read. Second, every model has a deputy, every deputy has a check, and the interesting question is always whether the check and the operation use the same identity and the same object.

The module in one sentence: enumerate the layers (uid sandbox, SELinux, permissions, verified boot, FBE, Keystore on Android; DAC, capabilities, LSM, namespaces, seccomp on Linux), read the policy as data, and then ask for every privileged operation which layer was supposed to stop it and which layer actually did. The gap between those two answers is the finding.

References

1. “The Confused Deputy”, Norm Hardy, ACM SIGOPS 1988
2. Linux man pages: `capabilities(7)`, `user_namespaces(7)`, `namespaces(7)`, `seccomp(2)`, `ptrace(2)`, `mount(2)`
3. `Documentation/userspace-api/seccomp_filter.rst` and `Documentation/security/lsm.rst` (kernel.org)
4. SELinux Project wiki (selinuxproject.org) and “SELinux by Example” (Mayer, MacMillan, Caplan)
5. Android Security documentation (source.android.com/security): sandbox, permissions, SELinux, verified boot, FBE, Keystore
6. Android Security Internals, Nikolay Elenkov (the Android model from the inside)
7. AOSP `system/sepolicy` and `frameworks/base` sources (android.googlesource.com)

8. setools (seinfo, sesearch, sediff) and audit2allow for reading and diffing a policy as data
9. Kernel Self Protection Project hardening recommendations (kspp) for the Linux-side defaults